Group 15 - Water Waves Report

Andreas Sinharoy (1804987),
Alex Gavriliu (1785060),
Vlad Chibulcutean (1780980)

A. Demonstrating ability to translate theory

For **dimensionality** and **homogeneity**, we have two inputs with units of metres, and an output in metres per second. To make the dataset dimensionally homogeneous, we can modify the inputs by multiplying wavelength and height by the gravitational constant, and then taking the square root of the result. That is, the new inputs will be $\sqrt{hg}$ and $\sqrt{\lambda g}$ instead of h and λ. This ensures that all input variables are expressed in the same units, ms^{-1}, facilitating better understanding and modelling. The units are ultimately ms^{-1} since the units of h and λ are m (metres) and the unit of g is ms^{-2}. Consequently, the units of hg and $h\lambda$ are $m \times \frac{m}{s^2} = \frac{m^2}{s^2}$, the square root of which (equivalent to the units of $\sqrt{hg}$ and $\sqrt{h\lambda}$) is $\frac{m}{s} = ms^{-1}$, or the unit of the speed. Thus, the modified inputs enable dimensional homogeneity in the problem, as the inputs and outputs will both have the same unit of ms^{-1}.

For scaling, we scaled the dataset (after it was made dimensionally homogeneous) by dividing the input (altered height and wavelength) and the output values by the output (velocity).This was done to adjust the values of our input data so they're all on the same scale. This helps our initial linear regression model work more effectively, and ensures that all input variables contribute equally to the learning process. Without scaling, variables with larger values might dominate the learning, leading to biases in the model.

Initial code for data cleaning and visualisation

```
[ ]  import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
```

```
[ ]  data = pd.read_csv("/water_waves_training_data.csv")
```

```
     g = 9.81
     hdata = data.copy()
     hdata['height'] = np.sqrt(g * hdata['height']) # height unit conversion to m/s
     hdata['wave_length'] = np.sqrt(g * hdata['wave_length']) # wave length unit conversion to m/s

     shdata = hdata.copy()
     shdata['height'] = shdata['height'] / shdata['speed']
     shdata['wave_length'] = shdata['wave_length'] / shdata['speed']
     shdata['speed'] = shdata['speed'] / shdata['speed']
```

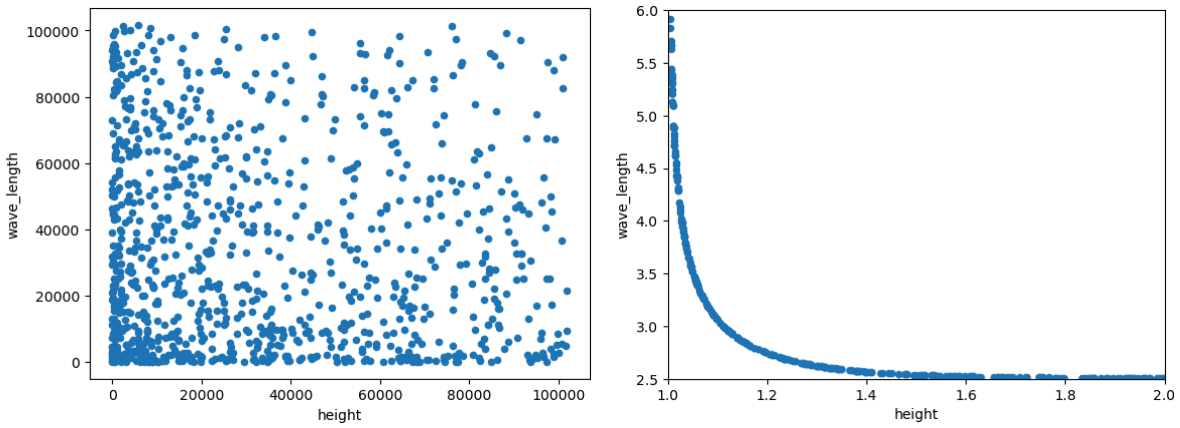
Above the code in the for the formatting the we import 3 libraries:

- **Pandas**, to make use of the Dataframe class so that we can easily store and format the wave data from the csv file in a convenient variable.
- **Numpy** provides many powerful tools and functions to perform mathematical operations and calculus on multidimensional arrays (Matrices) which we will make

great use of later when working with the modelling functions and their trainable parameters.

- **Matplotlip.pyplot**, so we can visualise our different sets of data.

Above we first initialise a dataframe containing all the raw wave data (*data*) from the csv file and make a copy of this dataFrame corresponding to the homogenous version of the data (*hdata*) where all input and output variables are of the same dimension and have the same yardstick. To do this we follow the line of thinking described previously by multiplying each previous input by **g** and square rooting it. From here to further normalise the data, for the sake of visualisation, we divide all the data in the table by **c** *(wave speed m/s)*. This way the value of the output column will be 1, and the data rather than representing some real physical value will be a relative change or deviation between values.



When visualising the dataset without considerations for scalability and dimensional homogeneity, the plot appears scattered or random, lacking any discernible pattern or relationship among the data points. This randomness arises due to the varying scales and dimensions of the input variables, which obscure any underlying structure in the data.

However, after scaling the data and ensuring dimensional homogeneity, the plot reveals a clear and coherent pattern. The data points align along a smooth curve, suggesting a systematic relationship between the input variables and the output. This coherent pattern indicates that the underlying physics of the problem can be accurately described by a single function.

“The physics of this problem is completely described by the 1-contour of a suitable function.”:

This sentence refers to the normalisation of the data to produce the data set where everything is divided by its respective output value on its row. This results in the entire output column having a value of 1 only and the other 2 inputs having some value that represents some change or deviation from their old value compared to the output. This is why it is referred to as a 1-contour as our output maintains a constant value of 1. This describes the “physics of the problem” because it produces a clear function or **contour** which we can mathematically approximate and analyse using a linear model. Allowing us to mathematically analyse the data through a model allows us to describe the physics based relationships that this data is representing.

B. Linear Regression and Generalised Linear Regression

First we implement a simple linear network with parameters 2 input variables and an array of weights that we can specify. We also add the value eps = 1e-16 as a parameter as this ensures the values are never exactly 0.

```
def linear_network(x0, x1, W, eps=1e-16):
    """Linear fit of a response variable dependent
    on explanatory variables x0, x1 weighted by W.
    """
    f = W[0]*x0 + W[1]*x1 + eps

    return x0/f, x1/f, f
```

This above function returns 3 values:

- $n = \frac{x_0}{x_0 W[0] + x_1 W[1] + eps}$
- $m = \frac{x_1}{x_0 W[0] + x_1 W[1] + eps}$
- $f = x_0 W[0] + x_1 W[1] + eps$

Then from here we check our network's accuracy in prediction using MAPE (Mean Absolute Percentage Error), and we also visualise these predicted values and display them side by side to better understand the optimal weights.

```
def fit_and_validate(weights, data, data_scaled, network):
    """
    input:
    weights      array of weights
    data         input data
    data_scaled  rescaled input data
    network      a function e.g. "linear_network" implemented above

    output: the MAPE value corresponding to the given input.
    """

    # Evaluate network and compute error
    n, m, f = network(data["height"], data["wave_length"], weights)
    mape    = 100 * np.mean(np.abs( f.values/data["speed"].values - 1))
```

In the method above, we initialise the network parameter with the modified but not normalised height, Wavelength, and some weights we assign. We also calculate based on the original and predicted data the MAPE using the formula above:

$$100 * \frac{1}{N} \sum_{i=1}^{N} \left| \frac{y_i - y^p{}_i}{y_i} \right|$$

```
        # plot scaled data
        ax1.scatter(data_scaled.height, data_scaled.wave_length, color='r', label='Data')
        # fitting on the scaled data
        ax1.scatter(n, m, marker="x", label="Prediction")
        ax1.legend()
        ax1.set_xlim(-5, 100)  # Limit x-axis to 0-200
        ax1.set_ylim(-5, 100)  # Limit y-axis to 0-200

        # verification on the actual data
        ax2.loglog(data.speed, f, marker="x", ls='', label="Prediction")
        # diagonal line
        z = [data.speed.min(), data.speed.max()]
        ax2.plot(z, z, color='r', label = "f(x,y)=d")
        # Report Error in the plot title
        ax2.set_title(f"MAPE = {mape:.2f} %" )
        ax2.set_xlabel("Reference")
        ax2.set_ylabel("Prediction")
        ax2.legend()

        return mape
```

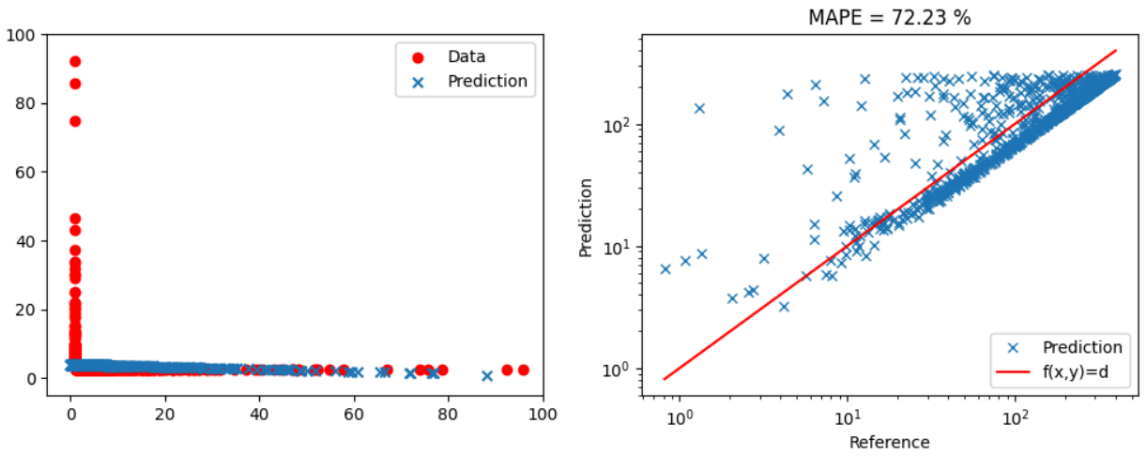
We then plot 2 graphs:

- The normalised data is then plotted for against the normalised prediction on one graph
- The raw data and prediction line are placed on a log'd graph.

```
W = [0.0085,0.25]
fit_and_validate(W, hdata, shdata, linear_network)
```

We then plug in the various data frames and the linear network as the parameters to obtain our visualisations:

```
72.23397248611882
```



As can be seen above trying to use a single line to predict the function on the left is not optimal as the contour seems to have a kink right at the start of the graph that split the data in half. So we obtain a very large MAPE.

A way to potentially improve the linear fit is by introducing a new input that uses the root of the product of both height (h) and wavelength (**λ**). This can then be multiplied by the acceleration of gravity, and then the product can be square rooted to obtain a new input which has the same unit of measurement and dimension as the output (m/s).

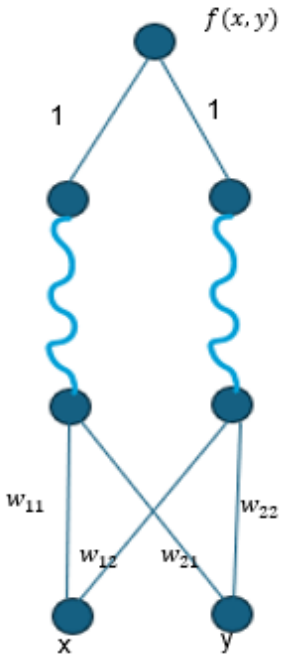
$$h * \lambda\,(m^2)\,,\ \sqrt{h\lambda}\,(m),\ g\sqrt{h\lambda}\,(m^2 s^{-2}),\sqrt{g\sqrt{h\lambda}}\,(ms^{-1})$$

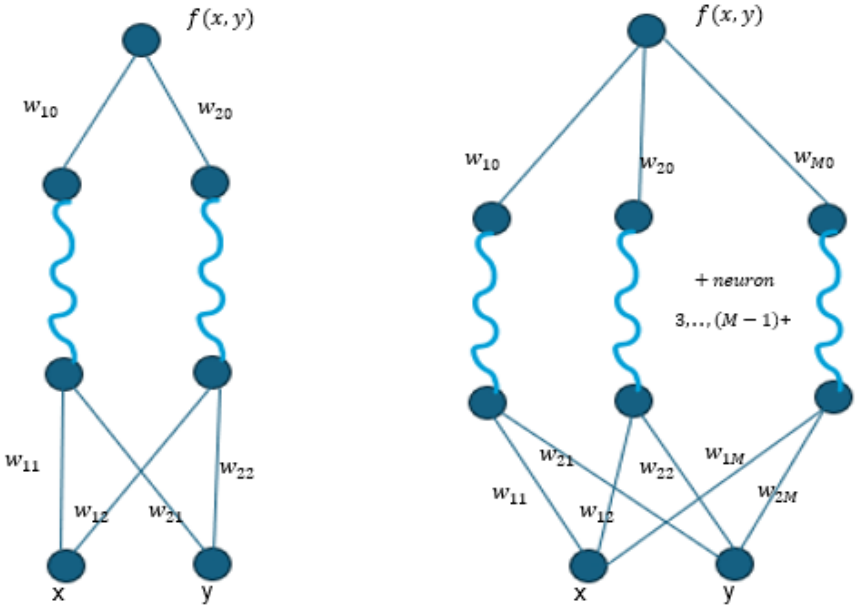
$\sqrt{g\sqrt{h\lambda}}$ and c have the same unit (m/s).

This can accurately depict the combined effect of these two factors thereby improving the model's accuracy. Another Input that can be altered to produce a more accurate fit is the specific values of gravity for each measurement of height and wavelength. Despite being small, the fluctuations in strength of gravity at different points of the planet does have an effect on the values measured for wavelength and height. Introducing these values into the model could further improve the accuracy and fit of the model.

C. Discuss how a shallow feed forward ReLU network could be used to model the data.

A shallow ReLU (Rectified Linear Unit) feed forward network is a structure which utilises as its activation function the ReLU function represented by $g(x) = max(0, x)$. The activation function's purpose here is to introduce a nonlinearity into a model in order to better fit more complex patterns. Below depicted are several of these structures:





First we have the top left diagram depicting the most basic ReLU implementation known as a **neuron**. This is represented by function $f(x,y) = g(w_1 x + w_2 y)$, and from this we can see the **neuron** gives us 2 parameters to fit our function. If we think about this visually, one neuron represents a single line segment, in this case being just one line, which we use the weights to try to fit to our desired function. Secondly we have 2 network diagrams each using 2 neurons (case A). Both of these connect these 2 neurons at their output using multipliers going to one node, forming a hidden layer of the original neuron's outputs. The function of the first diagram is: $f(x,y) = g(w_{11}x + w_{21}y) + g(w_{12}x + w_{22}y)$. These two end connecting multipliers have a weight of one just combining the 2 line segments the neurons form. The second of these 2 has function $f(x,y) = w_{10}\, g(w_{11}x + w_{21}y) + w_{20} g(w_{12}x + w_{22}y)$, where the connecting multipliers at the top of the neurons now have a specific assigned weight of w_{10}, w_{20} respectively. From this equation it is evident that the total number of trainable parameters available is **6** in total. Then lastly we have a ReLU feed forward of M neurons in total. The equation this yields is: $f(x,y) = w_{10}\, g(w_{11}x + w_{21}y) + w_{20} g(w_{12}x + w_{22}y) + \ldots + w_{M0} g(w_{1M}x + w_{1M}y)$. There are a total of **M** terms in this equation. Each term has a set of 3 trainable parameters, **1** on the outside of the bracket and **2** on the inside, hence there are a total of **3M** trainable parameters for this diagram. By adding neurons to these structures you are essentially dividing up the function you are modelling into multiple segments that form a curve so you can more finely control its shape and therefore its fit.
(Case A = 6 trainable parameters)
(Case B = 3M trainable parameters)

- Comment on the statement: *"Neural networks feature a universal approximation property"*.

A key idea in machine learning is the universal approximation feature of neural networks, which is that a neural network can approximate any continuous function with a suitable

number of neurons and activation functions. This characteristic highlights the adaptability and strength of neural networks in simulating intricate interactions in data across a range of fields. In theory, neural networks are able to learn and represent nearly any function. To fully utilise this approximation quality while minimising problems like overfitting, however, optimal performance frequently requires precise design considerations, such as architecture selection, regularisation strategies, and hyperparameter tuning.

- Given an annotated datasets, how would you train the neural network and check on the quality of its performance?

To train a neural network using annotated datasets, start by splitting the data into training and testing sets. Then, normalising or scaling the data if necessary for homogeneous inputs. Afterwards, choosing an appropriate neural network architecture, then training it on the training data using an optimization algorithm like SGD. Afterwards evaluating the model's performance on the testing set using metrics such as MSE or MAE. Finally, we should iterate through refining the model's architecture or adjusting hyperparameters until satisfactory performance is achieved.

- What would be an appropriate training loss, E?
  Please bear in mind your consideration on homogeneity and scaling.

An appropriate training loss E for a regression task such as ours could be mean squared error: $MSE = \frac{1}{N}\sum_{i=1}^{N} (y_i - y^p{}_i)^2$, where y_i is the actual target value and $y^p{}_i$ is the predicted value obtained from the model. MSE is a suitable training loss function for homogeneous and scaled data due to its characteristics aligning with the objectives of regression tasks. MSE penalises larger errors more significantly by squaring the errors between predicted and actual values, making it sensitive to deviations from the target. Moreover, since the data is homogeneous and scaled, MSE remains consistent across different input and output features and scales, ensuring fair comparison and optimisation. Lastly, its convexity ensures efficient optimisation algorithms, leading to quicker convergence towards a good solution.

- Can you write our shallow network in formulas using a matrix multiplication notation?

We can represent the functions/formulas that we have described previously as a matrix multiplication. Below is a representation of the $M \geq 2$ neuron network equation in matrix multiplication form:

$$f(x,y) = \begin{bmatrix} w_{10} & w_{20} & \dots & w_{M0} \end{bmatrix} g\left(\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ \dots & \dots \\ w_{1M} & w_{2M} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \right)$$

Here the ‘g()’ represents the ReLU function ($g(x) = max(0, x)$), and in this context g acts on each element of the matrix.

D. Gradient Descent and Backpropagation

- How is stochastic gradient descent (SGD) used to train neural networks? How is it different from gradient descent (GD)? In your answer, use the following equation to describe the differences between SGD and GD:

$$\Delta \mathbf{w} = -\mu \frac{\partial E}{\partial \mathbf{w}}. \tag{1}$$

- Calculate, in matrix form, the gradient of the network with respect to the weights in each linear layer. Use the backpropagation algorithm.
  Hint: take advantage of the lecture notes addendum.

SGD and GD are optimisation algorithms used to minimise the error of neural networks during training. The main difference between them is in their approach to actually changing the weights. GD computes the gradient of the error function with respect to all of the training samples, while SGD updates the weights using only one randomly chosen training sample at a time. This leads to faster convergence in SGD, with the method also being computationally cheaper since it processes individual samples rather than the full data. Nevertheless, its functioning can lead to more fluctuations during training as the randomness introduced by single-sample updates is not as smooth as GD.

Looking at the equation, Δw represents the change in weights, μ is the learning rate, and $\frac{\partial E}{\partial w}$ represents the gradient of the error with respect to the weights. For GD, the E, specifically, refers to the error function computed over the full training set. In contrast, for SGD, the E is changed to an E_i, which represents the error function computed for a single randomly chosen training sample i from the dataset. On top of these differences There is one distinct advantage that SGD has over GD thanks to its stochastic component. Weights when being calculated using GD can end up getting stuck at a local minimum or saddlepoints, as $\frac{\partial E}{\partial w}$ vanishes due to there being a flat gradient at this point. This causes the function to freeze and prevent further iteration as to leave this spot you must go in a direction directly opposing descent, which is problematic especially if there exist other more strong fitting values for said weights which are potentially lower than the minimum it is stuck at. SGD avoids this more often as it makes the change in weight values more random reducing the risk of being drawn to the less optimal local minimum value.

To calculate in matrix for the gradient of the network we have to use the backpropagation algorithm to obtain the formula we desire in matrix form. First we must calculate the derivative with respect to the output of each layer starting with the last output.

Let $\boldsymbol{x}_1, \ldots\ldots, \boldsymbol{x}_l$ denote the output variables of each layer where l is the last layer (network output). Starting with **x**l we first calculate :

$$\frac{\partial E}{\partial \boldsymbol{x}_l} = -(y - \boldsymbol{x}_l).$$

In this specific case the value is just the difference between the actual value and the predicted output. Also note that N_L denotes the number of nodes on layer l, and b is the number of batches. From here we then go to one layer below to calculate:

$$\frac{\partial E}{\partial \boldsymbol{x}_{l-1}}, \textit{which by reformulating with the chain rule can give us:}$$

$$\frac{\partial E}{\partial \boldsymbol{x}_{l-1}} = \frac{\partial E}{\partial \boldsymbol{x}_l} \frac{\partial \boldsymbol{x}_l}{\partial \boldsymbol{x}_{l-1}}$$

These 2 smaller derivatives can be more easily obtained and used to get our full derivative. Now depending on whether this layer is a linear operation (1), or ReLU activation (2) we do the following:

(1) If the *l*th layer is linear then $\boldsymbol{x}_l = \boldsymbol{x}_{l-1}\boldsymbol{W}_l$

$$so: \frac{\partial E}{\partial \boldsymbol{x}_{l-1}} = \frac{\partial E}{\partial \boldsymbol{x}_l}\frac{\partial \boldsymbol{x}_{l-1}\boldsymbol{W}_l}{\partial \boldsymbol{x}_{l-1}} = \frac{\partial E}{\partial \boldsymbol{x}_l}\boldsymbol{W}_l$$

(2) If the lth layer is a ReLU activation then $\boldsymbol{x}_l = ReLU(\boldsymbol{x}_{l-1})$

Since , $\frac{\partial ReLU(x)}{\partial x} = Heaviside(x)$ we can then deduce:

$$so: \frac{\partial E}{\partial \boldsymbol{x}_{l-1}} = \frac{\partial E}{\partial \boldsymbol{x}_l}\frac{\partial ReLU(\boldsymbol{x}_{l-1})}{\partial \boldsymbol{x}_{l-1}} = \frac{\partial E}{\partial \boldsymbol{x}_l} \otimes Heaviside(\boldsymbol{x}_{l-1})$$

Because of the fact we can use the chain rule between a layer $\boldsymbol{x}_l$ and the one below $\boldsymbol{x}_{l-1}$,we can repeat this process all the way back through the network to calculate all of these gradients.

Once we have calculated the entire network of gradients we can now start calculating the change in the gradient representing the change in the weights, where l represents the layer being dealt with:

$$\Delta \boldsymbol{w}_l = \frac{\partial E}{\partial \boldsymbol{W}_l}$$

Once again using the chain rule we can manipulate this gradient to obtain:

$$\frac{\partial E}{\partial \boldsymbol{W}_l} = \frac{\partial E}{\partial \boldsymbol{x}_l}\frac{\partial \boldsymbol{x}_l}{\partial \boldsymbol{W}_l}$$

This first term has already been computed, so we only need to further algebraically manipulate the second term. We already know for a fact that $\boldsymbol{x}_l = \boldsymbol{x}_{l-1}\boldsymbol{W}_l$ is true for each linear layer, where the weights are present. So we can derive:

$$\frac{\partial E}{\partial \boldsymbol{W}_l} = \frac{\partial E}{\partial \boldsymbol{x}_l}\frac{\partial \boldsymbol{x}_l}{\partial \boldsymbol{W}_l} = \frac{\partial E}{\partial \boldsymbol{x}_l}\frac{\partial \boldsymbol{x}_{l-1}\boldsymbol{W}_l}{\partial \boldsymbol{W}_l} = \frac{\partial E}{\partial \boldsymbol{x}_l}\boldsymbol{x}_{l-1}$$

So we end with: $\Delta \boldsymbol{w}_l = \frac{\partial E}{\partial \boldsymbol{W}_l} = \frac{\partial E}{\partial \boldsymbol{x}_l}\boldsymbol{x}_{l-1}$

E. Implementation of a Shallow Feed Forward Net:

Firstly, we created a training-testing split of the data with a 80/20 split, and created two dataframes train_data and test_data.

```
# Create training and testing data split
from sklearn.model_selection import train_test_split
X = hdata[['height', 'wave_length']]
y = hdata['speed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train.reset_index(drop=True, inplace=True)
X_test.reset_index(drop=True, inplace=True)
y_train.reset_index(drop=True, inplace=True)
y_test.reset_index(drop=True, inplace=True)

train_data = pd.concat([X_train, y_train.to_frame()], axis=1)
test_data = pd.concat([X_test, y_test.to_frame()], axis=1)
```

Next, we created variables for all of the relevant values in our model.

```
nr_neurons = 1000

nr_epochs = 10000

learning_rate = 0.000001

nr_batches = 5

batch_size = len(train_data) / nr_batches

nr_inputs = 2

nr_weights = nr_neurons * nr_inputs + nr_neurons
```

Note that the actual values shown above for the variables are not necessarily the ones that we used to get our best models, as these were played with and changed many times in order to improve the accuracy of the model. Nevertheless, these were the variables used. $nr\_neurons$ represents the number of neurons in the hidden layer, nr_epochs represents the number of times that the model goes through the entire dataset, $learning\_rate$ represents the learning rate used to compute the gradient of descent, $nr\_batches$ represents the number of batches based on which the gradient of descent is computed, nr_inputs represents the number of inputs going into the model initially, and $nr\_weights$ represents the current number of weights in the entire model.

Once the variables were created, we went onto defining functions that our model would later use.

```
def initialise_weights(nr_inputs, nr_neurons):
  nr_weights1 = nr_inputs * nr_neurons
  nr_weights2 = nr_neurons
  return np.random.uniform(-1, 1, size=nr_weights1), np.random.uniform(-1, 1, size=nr_weights2)

def relu(x):
  if (x > 0):
    return x
  else:
    return 0

def heavyside(x):
  if (x >= 0):
    return 1
  else:
    return 0

def split_dataframe(df, num_splits):
    # Shuffle the rows of the DataFrame randomly
    shuffled_df = df.sample(frac=1).reset_index(drop=True)

    # Calculate the number of rows in each split
    rows_per_split = len(df) // num_splits

    # Split the DataFrame into smaller DataFrames
    split_dfs = []
    start_idx = 0
    for i in range(num_splits):
        end_idx = start_idx + rows_per_split
        if i == num_splits - 1:  # For the last split, include remaining rows
            end_idx = len(df)
        split_dfs.append(shuffled_df.iloc[start_idx:end_idx])
        start_idx = end_idx
    return split_dfs
```

$initialise\_weights$ was created to initialise the weights to a random value between -1 and 1, $relu$ to calculate the ReLU values for rectifiers in the model, $heavyside$ for calculating the δ values for backpropagation, and $split\_dataframe$ to split the dataset into batches prior to every epoch.

Further helper methods that we created are shown below:

```
def reshape_weights1(weights, nr_neurons, nr_inputs):
    if len(weights) != nr_neurons * nr_inputs:
        raise ValueError("Size of weights list does not match given dimensions.")

    # Reshape the weights list into a matrix
    weights_matrix = np.array(weights).reshape(nr_inputs, nr_neurons)

    return weights_matrix

def calculate_neurons(weights1, weights2, inputs):
  neurons = []
  inputs = np.array(inputs)
  inputs = inputs.reshape(1, nr_inputs)
  weights1 = reshape_weights1(weights1, nr_neurons, nr_inputs)
  hidden_neurons = (inputs@weights1)[0]
  relu_neurons = []
  for i in range(len(hidden_neurons)):
    relu_neurons.append(relu(hidden_neurons[i]))
  output_neuron = 0
  for i in range(nr_neurons):
    output_neuron += relu_neurons[i] * weights2[i]
  return np.concatenate((hidden_neurons, relu_neurons, [output_neuron]))

def calc_ape(neuron_values, output_value):
  ape = (neuron_values[-1] - output_value)
  return ape

def backprog(weights1, weights2, inputs, neuron_values, output_value):
  gradient_desc = []
  deltas = []
  deltas.append((neuron_values[-1] - output_value))
  for i in range(nr_neurons):
    deltas.append((deltas[0]) * weights2[nr_neurons - (1 + i)])
  for i in range(nr_neurons):
    deltas.append(deltas[i + 1] * heavyside(neuron_values[nr_neurons - (1 + i)]))
  deltas = deltas[::-1]
  for i in range(nr_inputs):
    for j in range(nr_neurons):
      gradient_desc.append(learning_rate * deltas[j] * inputs[i])
  for i in range(nr_neurons):
    gradient_desc.append(learning_rate * deltas[-1] * neuron_values[i + nr_neurons])
  return gradient_desc
```

The $reshape\_weights1$ method is used by the $calculate\_neurons$ method to make the weights array into a matrix of appropriate dimension for the matrix multiplication. $calculate\_neurons$ calculates the values of all neurons in the network given an input and the weights. Note: $weights1$ represents the weights of the hidden layer, whilst $weights2$ represent the weights from the activation functions to the output. $calc\_ape$ calculates the percentage error of the model for each input, aiding in the evaluation of the model. Lastly, the $backprog$ method is in charge of carrying out the backpropagation, ultimately returning the gradient of descent.

```
weights1, weights2 = initialise_weights(nr_inputs, nr_neurons)
for i in range(nr_epochs):
  print(i)
  batch_mapes = []
  epoch_mape = 0
  batches = split_dataframe(train_data, nr_batches)
  for i in range(nr_batches):
    gradients = []
    apes = []
    mape = 0
    for index, row in batches[i].iterrows():
      inputs = row.loc[['height', 'wave_length']].tolist()
      output = row.loc[['speed']].tolist()[0]
      neurons_i = calculate_neurons(weights1, weights2, inputs)
      apes.append(calc_ape(neurons_i, output))
      gradient_desc = backprog(weights1, weights2, inputs, neurons_i, output)
      gradients.append(gradient_desc)
    final_gradients = [0] * nr_weights
    for i in range(len(apes)):
      mape += apes[i]
    batch_mapes.append(mape/batch_size)
    for list in gradients:
      for i in range(len(list)):
        final_gradients[i] += list[i]
    for i in range(len(final_gradients)):
      final_gradients[i] = final_gradients[i] / (batch_size)
    for i in range(nr_weights):
      if (i < nr_neurons*nr_inputs):
        weights1[i] -= final_gradients[i]
      else:
        weights2[i - nr_neurons*nr_inputs] -= final_gradients[i]
  for i in range(len(batch_mapes)):
    epoch_mape += batch_mapes[i]
  epoch_mape = epoch_mape / nr_batches
  print(epoch_mape * 100)
  if abs(epoch_mape) < abs(lowest_mape):
      lowest_mape = epoch_mape
      lowest_weights = np.concatenate((weights1, weights2))
weights1, weights2
```

The above code actually runs everything. The weights are initialised, and then the first epoch begins. Within each epoch, the batches are initialised, and the data points in the batches start to be considered sequentially. The neurons are calculated for each input data point given the current weights, and then backpropagation is called to calculate the recommended change in weights for that input. Then, for all inputs in that batch, the average of the recommended change in weights is taken, and that average is the gradient of descent. Subsequently, the weights are updated according to this gradient of descent. Also, each epoch is printed for the coders to keep track, and the MAPE for each epoch is also printed for the coders to see

whether the model is being improved. Once all of the runs of the epoch are completed, the final MAPE is calculated and the model prediction is carried out on the verification data after the data is retrieved, as shown below:

```
ape = []
mape = 0
counter = 0
for index, row in test_data.iterrows():
    inputs = row.loc[['height', 'wave_length']].tolist()
    output = row.loc[['speed']].tolist()[0]
    neurons = calculate_neurons(weights1, weights2, (inputs))
    ape.append(calc_ape(neurons, output))
    print(neurons[-1], output)

for i in range(len(ape)):
    mape += ape[i]
    counter+=1
mape = mape/counter
print(mape)


verification_data = pd.read_csv("water_waves_verification_data_input.csv")
verification_data['h'] = np.sqrt(g * verification_data['h']) # height unit conversion to m/s
verification_data['lam'] = np.sqrt(g * verification_data['lam']) # wave length unit conversion to m/s
prediction_data = verification_data[['id']]
prediction_data['Expected'] = 0
verification_data

for index, row in verification_data.iterrows():
    inputs = row.loc[['h', 'lam']].tolist()
    neurons = calculate_neurons(weights1, weights2, (inputs))
    prediction_data.at[index, 'Expected'] = neurons[-1]
prediction_data

prediction_data['Id'] = prediction_data['id']
prediction_data = prediction_data[['Id','Expected']]

csv_file_path = 'prediction_data.csv'
# Use the to_csv() method to save the DataFrame as a CSV file
prediction_data.to_csv(csv_file_path, index=False)
```

The first piece of code computes and prints the final MAPE, for the coders. Then, the verification_data is obtained, made homogeneous, and set up in the desired format. Finally, the model is used to predict the outputs based on the verification_data. The predictions are then obtained, and along with ID's for each prediction, a double nested array is created with the predictions and the IDs. Then the array is converted to a csv file, and the results are uploaded to kaggle.